

Guía Completa de Uso de Git y GitHub

1. Descarga e Instalación de Git

Antes de poder utilizar cualquier comando de control de versiones, necesitamos tener instalado **Git** en nuestro sistema. Git es el "motor" local que funciona en nuestro ordenador, mientras que **GitHub** es la plataforma en la nube donde alojamos el código.

Instalación en Windows

1. Dirígete a la página oficial de Git: <https://git-scm.com/download/win>.
 2. Descarga el instalador de **64-bit Git for Windows Setup**.
 3. Ejecuta el archivo `.exe` descargado. Durante la instalación, puedes dejar todas las opciones por defecto haciendo clic en *Next* hasta terminar.
 4. Para comprobar que se ha instalado correctamente, abre tu terminal (PowerShell o CMD) y escribe: `bash`
`git --version` Si responde con la versión de Git (ej. `git version 2.45.0.windows.1`), la instalación ha sido un éxito.
-

2. Configuración Inicial (Primeros Pasos)

Recién instalado, Git no sabe quién eres. Cada vez que guardes un cambio, Git necesita firmarlo con tu nombre y correo. Esto solo debes hacerlo **una vez** en tu ordenador.

Abre la terminal y ejecuta los siguientes comandos (cambiando los datos por los tuyos):

```
# Configura tu nombre completo
git config --global user.name "Fran"

# Configura tu correo electrónico (usa el mismo con el que te
registraste en GitHub)
git config --global user.email "tu_correo@ejemplo.com"
```

3. Preparando el Proyecto para subirlo a GitHub

Si tienes la carpeta de tu proyecto (EuroCoinCollector) y aún no está conectada a GitHub, debes seguir estos tres pasos iniciales:

Paso A: Iniciar Git en tu carpeta

Abre la terminal en la raíz de tu proyecto y escribe:

```
git init
```

Esto crea una carpeta oculta `.git` que convierte tu carpeta normal en un repositorio.

Paso B: Crear el repositorio en la web de GitHub

1. Ve a [GitHub.com](https://github.com) e inicia sesión.
2. Arriba a la derecha, haz clic en el botón "+" y selecciona **New repository**.
3. Ponle el nombre EuroCoinCollector , déjalo público o privado según desees y **no marques** la opción de añadir un README (ya que ya hemos creado uno localmente).
4. Haz clic en **Create repository**. GitHub te dará una URL (ej.
`https://github.com/tu-usuario/EuroCoinCollector.git`).

Paso C: Conectar el local con la nube

Vuelve a la terminal en tu ordenador (donde hiciste `git init`) y vincula tu carpeta con el repositorio web:

```
# Preparamos todos los archivos para el primer guardado
git add .

# Hacemos nuestro primer guardado local
git commit -m "feat: Primer commit del proyecto completo con Docker
y README"

# Cambiamos el nombre de la rama principal de 'master' a 'main'
(estándar actual)
git branch -M main

# Conectamos nuestra carpeta local con el servidor de GitHub
(sustituye la URL)
git remote add origin https://github.com/tu-
usuario/EuroCoinCollector.git

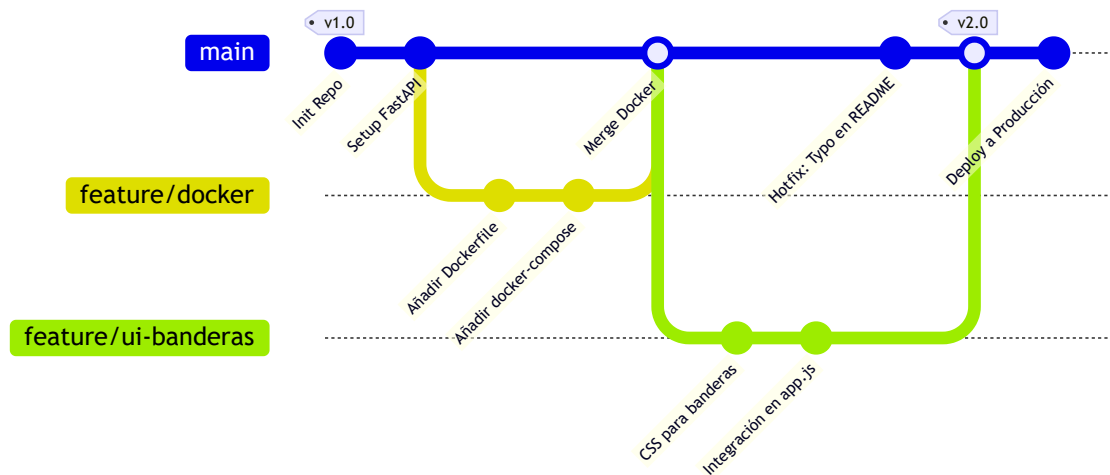
# Subimos todos los archivos a GitHub
git push -u origin main
```

***Nota:** La primera vez que hagas `git push`, GitHub te pedirá que inicies sesión en una ventana emergente en tu navegador para dar permisos a tu terminal.*

4. Flujo de Trabajo (Workflow) Recomendado

En este proyecto, seguimos un modelo simplificado basado en **Feature Branches** (Ramas de Características). Nunca trabajamos directamente sobre la rama principal (`main`), sino que creamos una rama paralela.

Diagrama de Flujo de Ramas



Explicación del Flujo

1. **main** : Es la fuente de la verdad. El código aquí siempre debe funcionar y estar listo para producción.
2. **feature/*** : Para cada nueva funcionalidad, creamos una rama. Aquí experimentamos y construimos.
3. **Merge**: Cuando la funcionalidad está terminada y probada, la integramos de vuelta a **main**.

5. Comandos Diarios Esenciales

5.1. Obtener actualizaciones

Antes de empezar a programar, siempre debemos descargar los últimos cambios de GitHub para evitar conflictos.

```
# Descarga los cambios y los fusiona directamente en tu rama actual
git pull origin main
```

5.2. Crear una nueva rama

```
# Crea una rama llamada 'feature/dark-mode' y te mueve a ella
```

```
git checkout -b feature/dark-mode
```

5.3. Guardar el trabajo (Commit)

```
# Añade todos los archivos modificados
git add .

# Crea el paquete con un mensaje descriptivo
git commit -m "feat(ui): Añadir variables de color para dark mode"
```

5.4. Subir a la nube (Push)

```
# Sube tu rama a GitHub
git push origin feature/dark-mode
```

6. Buenas Prácticas y Convenciones

Para mantener un historial limpio y profesional, el proyecto se adhiere a la especificación **Conventional Commits**.

Tipos soportados: - feat : Una nueva característica (ej. feat(api): endpoint de exportación a CSV) - fix : Corrección de un bug (ej. fix(ui): botón de eliminar no respondía en móviles) - docs : Cambios en la documentación (ej. docs: actualizar diagrama de arquitectura)

Archivo .gitignore

En nuestro repositorio verás un archivo llamado .gitignore . Aquí listamos carpetas y archivos que Git **jamás** debe subir a internet (como la carpeta venv/ , __pycache__/ o el archivo con las contraseñas de la base de datos .env).

7. Conclusión de la Guía

Git no es un sistema de copias de seguridad estático, es una máquina del tiempo y una herramienta de colaboración. Al seguir este flujo desde su correcta instalación y configuración inicial, aseguramos que el código de **EuroCoinCollector** esté a salvo en la nube de GitHub, y que el proyecto se mantenga ordenado y preparado para el trabajo profesional.